فیلسوف i فقط در صورتی می‌تواند state[i] = EATING انتخاب کند که هر دو همسایه‌اش در حال خوردن نباشند یعنی:

```
(state[(i+4) % 5] != EATING) && (state[(i+1) % 5] != EATING)
```

همین‌طور لازم داریم که اعلانی به صورت زیر داشته باشیم:

```
condition self[5];
```

با این شیوه، فیلسوف i موقعی که گرسنه است، اما قادر به برداشتن چوبک‌های مورد نیازش نیست، می‌تواند خود را به تأخیر بیندازد.

هم‌اکنون آمادگی داریم تا راه‌حل خود را برای مسأله‌ی فیلسوفان خوش‌خوراک تشریح کنیم. توزیع چوبک‌ها توسط مانیتور DiningPhilosophers که تعریف آن در شکل ۷-۷ نشان داده شده است، کنترل می‌شود. هر فیلسوف، پیش از آن که شروع به خوردن کند بایستی عملیات pickup() را احضار نماید. این کار ممکن است موجب تعلیق پردازه‌ی فیلسوف گردد. فیلسوف با تکمیل موفقیت‌آمیز عملیات خود می‌تواند خوردنش را شروع کند. در ادامه، فیلسوف عملیات putdown() را احضار می‌کند. بنابراین، فیلسوف i بایستی عملیات pickup() و putdown() را به صورت دنباله‌ی زیر احضار نماید:

```
DiningPhilosophers.pickup(i);
                ...
                eat
                ...
DiningPhilosophers.putdown(i);
```

به راحتی می‌توان نشان داد که این راه‌حل تضمین می‌کند هیچ دو همسایه‌ای نمی‌توانند همزمان مشغول خوردن باشند و هیچ‌گونه بن‌بستی اتفاق نخواهد افتاد. با این وجود، توجه داریم که همچنان امکان مرگ یک فیلسوف در مواجهه با قحطی‌زدگی وجود دارد. راه‌حلی برای این مسأله ارائه نمی‌کنیم، بلکه آن را به عنوان یک تمرین برای شما واگذار می‌کنیم.

۷-۲ همگامی در میان هسته

در ادامه، راهکارهای همگامی را که توسط سیستم‌عامل‌های ویندوز و لینوکس فراهم می‌شوند، مطرح می‌کنیم. این دو سیستم‌عامل، مثال‌های خوبی از رویکردهای متفاوت همگامی هسته فراهم می‌کنند و همان‌گونه که خواهید دید راهکارهای همگامی موجود در این سیستم‌ها، تفاوت‌های ظریف و قابل توجهی دارند.

۷-۲-۱ همگامی در ویندوز

سیستم‌عامل ویندوز یک هسته‌ی چندنخی است که پشتیبانی برنامه‌های کاربردی بلادرنگ و

پردازنده‌های متعدد را فراهم می‌کند. موقعی که هسته‌ی ویندوز یک منبع سراسری را روی یک سیستم تک‌پردازنده دستیابی می‌کند، موقتاً وقفه‌های تمامی روال‌های راه‌انداز وقفه را نیز که ممکن است منبع سراسری را دسترسی نمایند، منع می‌نماید. ویندوز در یک سیستم چندپردازنده دسترسی منابع سراسری را با استفاده از قفل‌های چرخشی، هرچند که هسته از قفل‌های چرخشی فقط برای حفاظت قطعات کد کوتاه استفاده می‌کند، محافظت می‌کند. به علاوه، هسته به دلایل کارآمدی اطمینان می‌دهد که نخی که قفل‌چرخشی در اختیار دارد، هرگز پس‌گرفته نخواهد شد.

```
monitor DiningPhilosophers
{
        enum {THINKING, HUNGRY, EATING} state[5];
        condition self[5];
        void pickup(int i) {
                state[i] = HUNGRY;
                test(i);
                if (state[i] != EATING)
                        self[i].wait();
        }
        void putdown(int i) {
                state[i] = THINKING;
                test((i + 4) % 5);
                test((i + 1) % 5);
        }
        void test(int i) {
        if ((state[(i + 4) % 5] != EATING) && (state[i] == HUNGRY) &&
                (state[(i + 1) % 5] != EATING)) {
                        state[i] = EATING;
                        self[i].signal();
                }
        }
        initialization_code() {
        for (int i = 0; i < 5; i++)
                state[i] = THINKING;
        }
}
```

شکل ۷-۷ راه‌حل مانیتوری برای مسأله‌ی فیلسوفان خوش‌خوراک.

ویندوز برای همگامی نخ در بیرون هسته، **اشیاء اعزام‌کننده**[1] را فراهم می‌کند. نخ‌ها با استفاده از شئ اعزام‌کننده، مطابق چندین راهکار متفاوت، اعم از قفل‌های انحصار متقابل، سمافورها،

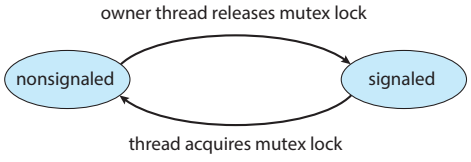
1. Dispatcher objects

رویدادها و تایمرها همگام می‌شوند. سیستم، داده‌های مشترک را با الزام یک نخ برای کسب مالکیت یک انحصار متقابل و رهایی مالکیت آن به محض اتمام کار محافظت می‌کند. سمافورها آن‌گونه که در بخش ۶-۶ مطرح شدند، رفتار می‌کنند. **رویدادها** مشابه متغیرهای شرط هستند؛ یعنی، نخ منتظری را در زمان برقراری یک شرط مطلوب مطلع می‌سازند. بالاخره، تایمرها جهت اطلاع‌رسانی یک (یا بیشتر) نخ مبنی بر پایان و سپری‌شدن یک مدت زمان مشخص استفاده می‌شوند.

اشیاء اعزام‌کننده می‌توانند در حالت سیگنال‌دار [1] و یا حالت بی‌سیگنال [2] قرار بگیرند. یک شئ در **حالت سیگنال‌دار** در دسترس است و هر زمان‌که نخی بخواهد شئ را کسب کند، مسدود نخواهد شد. یک شئ در **حالت بی‌سیگنال** در دسترس نیست و اگر نخی بخواهد شئ را به دست آورد، مسدود خواهد شد. انتقال حالت‌های شئ اعزام‌کننده‌ی یک قفل انحصار متقابل را در شکل ۷-۸ نمایش می‌دهیم.

رابطه‌ای میان حالت یک شئ اعزام‌کننده و حالت یک نخ وجود دارد. زمانی که یک نخ بر روی یک شئ اعزام‌کننده‌ی بی‌سیگنال مسدود می‌شود، حالت آن از آماده به انتظار تغییر می‌یابد و نخ در صف انتظار مربوط به شئ قرار داده می‌شود. لحظه‌ای که حالت شئ اعزام‌کننده به سیگنال‌دار جابه‌جا می‌شود، هسته بررسی می‌کند تا ببیند آیا نخی به انتظار شئ وجود دارد یا خیر. اگر چنین بود، هسته نخ (یا در صورت امکان، نخ‌های بیشتری) را از حالت انتظار به حالت آماده می‌آورد تا در آنجا بتواند اجرای خود را از سر بگیرد. تعداد نخ‌هایی که هسته از صف انتظار انتخاب می‌کند به نوع شئ اعزام‌کننده که آنها برای آن منتظرند، بستگی دارد. هسته برای انحصار متقابل فقط یک نخ را از صف انتظار انتخاب خواهد کرد، چون شئ انحصار متقابل فقط می‌تواند در «مالکیت» یک نخ منفرد قرار بگیرد. برای یک شئ رویداد، هسته تمامی نخ‌های منتظر آن رویداد را انتخاب خواهد کرد.

برای روشن‌کردن اشیاء اعزام‌کننده و حالت‌های نخ می‌توانیم از قفل انحصار متقابل استفاده کنیم. اگر نخی بخواهد یک شئ اعزام‌کننده را که در حالت بی‌سیگنال می‌باشد، کسب کند، آن نخ معلق خواهد شد و در صف انتظار شئ انحصار متقابل گذاشته می‌شود. لحظه‌ای که انحصار متقابل به حالت سیگنال‌دار جابه‌جا می‌شود (چون نخ دیگری، قفل روی انحصار متقابل را رها می‌کند)، نخ منتظر جلوی صف از حالت انتظار به حالت آماده جابه‌جا شده و قفل انحصار متقابل را کسب خواهد کرد.



شکل ۷-۸ شئ اعزام‌کننده‌ی انحصار متقابل.

شئ بخش بحرانی، انحصار متقابل مد کاربری است که غالباً می‌تواند بدون مداخله‌ی هسته کسب و رها شود. در سیستم چندپردازنده، شئ بخش بحرانی نخست در حالی‌که به انتظار رهایی

1. Signaled state
2. Nonsignaled state

شئ توسط نخ دیگر است از قفل چرخشی استفاده می‌کند. اگر چرخش آن زیادی طول بکشد، در ادامه نخی که به دنبال کسب شئ است، یک انحصار متقابل هسته در اختیار گرفته و پردازنده‌اش را رها می‌کند. اشیاء بخش بحرانی مخصوصاً کارآمد هستند، چون انحصار متقابل هسته فقط موقعی تخصیص می‌یابد که برای شئ رقابت وجود دارد. در عمل، رقابت بسیار کمی وجود دارد، لذا صرفه‌جویی‌ها درخور هستند.

در انتهای این فصل، یک پروژه‌ی برنامه‌نویسی که از قفل‌های انحصار متقابل و سمافورهای API ویندوز استفاده می‌کند تدارک دیده‌ایم.

۷-۲-۲ همگامی در لینوکس

لینوکس، قبل از نسخه‌ی 2.6، هسته‌ای غیرقابل پس‌گیر بود، به این معنی که اگر پردازه‌ای در مد هسته اجرا داشت نمی‌توانست پس‌گرفته شود—حتی اگر پردازه‌ای با الویت بالاتر هم برای اجرا آماده می‌شد. با این وجود، هسته‌ی کنونی لینوکس کاملاً پس‌گرفتنی است، لذا وظیفه‌ای که در هسته در حال اجراست، می‌تواند پس گرفته شود.

لینوکس چندین راهکار متفاوت برای همگامی در هسته فراهم می‌کند. همان‌گونه که اکثر معماری‌های کامپیوتر، دستورالعمل‌هایی برای نسخه‌های یکپارچه‌ی عملیات ریاضی ساده فراهم می‌کنند، ساده‌ترین تکنیک همگامی در میان هسته‌ی لینوکس، عدد صحیح یکپارچه است که با استفاده از نوع داده‌ی مبهم atomic_t نمایش داده می‌شود. همان‌گونه که از اسم آن بر می‌آید، تمامی عملیات ریاضی که از اعداد صحیح یکپارچه استفاده می‌کنند، بدون وقفه انجام می‌گیرند. برای روشن شدن مطلب، برنامه‌ای را در نظر بگیرید که شامل یک counter صحیح و یک value صحیح یکپارچه می‌باشد.

```
atomic_t counter;
int value;
```

کد زیرین اثر انجام عملیات یکپارچه‌ی مختلف را نمایش می‌دهد:

Atomic Operation	*Effect*
`atomic_set(&counter,5);`	`counter = 5`
`atomic_add(10,&counter);`	`counter = counter + 10`
`atomic_sub(4,&counter);`	`counter = counter - 4`
`atomic_inc(&counter);`	`counter = counter + 1`
`value = atomic_read(&counter);`	`value = 12`

متغیرهای صحیح یکپارچه مخصوصاً در شرایطی که نیاز است تا یک متغیر صحیح–مثل شمارنده–به روز شود، کارآمد هستند، چرا که عملیات یکپارچه سربار راهکارهای قفل‌گذاری را لازم ندارند. با این وجود، کاربرد آنها تنها به مواردی از این دست محدود می‌شود. در شرایطی که متغیرهای متعددی برای یک شرط رقابت ممکن سهیم هستند، بایستی ابزار قفل‌گذاری پیچیده‌تری استفاده شوند.

قفل‌های انحصار متقابل لینوکس با هدف حفاظت بخش‌های بحرانی میان هسته وجود دارند. در اینجا، یک وظیفه بایستی قبل از ورود به بخش بحرانی، تابع mutex_lock() و بعد از خروج از

بخش بحرانی، تابع ()mutex_unlock را احضار کند. اگر قفل انحصار متقابل در دسترس نباشد، وظیفه‌ای که ()mutex_lock را فراخوانی کرده، در یک حالت خواب گذاشته می‌شود و موقعی که مالک قفل، تابع ()mutex_unlock را احضار می‌کند از خواب بیدار می‌گردد.

همین‌طور، هسته‌ی لینوکس، قفل‌های چرخشی و سمافورها را (در کنار نسخه‌های خواننده–نویسنده‌ی این دو قفل) برای قفل‌گذاری در داخل هسته فراهم می‌کند. راهکار اساسی قفل‌گذاری ماشین‌های SMP، قفل چرخشی است و هسته طوری طراحی می‌شود که قفل چرخشی فقط برای فواصل زمانی کوتاهی نگه داشته شود. استفاده از قفل‌های چرخشی در ماشین‌های تک‌پردازنده، از قبیل سیستم‌های تعبیه‌شده که فقط یک هسته‌ی پردازشی منفرد دارند، مناسب نبوده و با فعال و غیرفعال‌کردن پس‌گیری هسته جایگزین می‌شوند. یعنی، در ماشین‌های تک‌پردازنده، هسته به جای نگه‌داری قفل چرخشی، پس‌گیری هسته را غیرفعال می‌کند؛ و به جای رهایی قفل چرخشی، پس‌گیری هسته را فعال می‌کند. این عملیات در زیر خلاصه می‌شود:

چند پردازنده ها	تک پردازنده
قفل چرخشی را در اختیار بگیر	پس گیری هسته را غیرفعال کن
قفل چرخشی را رها کن	پس گیری هسته را فعال کن

قفل‌های چرخشی و قفل‌های انحصار متقابل در لینوکس، هر دو *غیربازگشتی* هستند، این موضوع به این معنی است که اگر یک نخ، یکی از این قفل‌ها را در اختیار گرفت، نمی‌تواند همان قفل را برای بار دوم بگیرد بدون آن که اول قفل را رها کند. در غیر این صورت، تلاش بار دوم در کسب قفل مسدود خواهد شد.

لینوکس از رویکرد جالبی برای فعال و غیرفعال‌کردن پس‌گیری هسته استفاده می‌کند. برای این کار، دو فراخوانی سیستمی ساده—()preempt_disable و ()preempt_enable—برای غیرفعال و فعال‌کردن پس‌گیری هسته فراهم می‌نماید. با این وجود، اگر یک وظیفه با اجرا در هسته قفلی را نگه داشته باشد، هسته پس‌گرفتنی نیست. برای اعمال این قانون، هر وظیفه در سیستم، یک رکورد thread_info شامل یک شمارنده به نام preempt_count دارد که تعداد قفل‌های در اختیار وظیفه را نشان می‌دهد. هر زمان که قفلی کسب می‌شود، preempt_count یک واحد افزایش می‌یابد. با رهایی یک قفل نیز یک واحد کاهش می‌یابد. اگر مقدار preempt_count وظیفه‌ای که فعلاً در هسته اجرا دارد بزرگ‌تر از ۰ باشد، پس‌گیری هسته ایمن نیست، چرا که این وظیفه فعلاً قفلی را نگه داشته است. اگر شمارش ۰ باشد، هسته می‌تواند به صورت ایمن متوقف شود (با این فرض که، هیچ فراخوانی برای ()preempt_disable نمانده است).

قفل‌های چرخشی—به همراه فعال و غیرفعال‌کردن پس‌گیری هسته—تنها زمانی در هسته استفاده می‌شوند که قفل (یا غیرفعال‌کردن پس‌گیری هسته) برای فاصله‌ی زمانی کوتاهی نگه داشته شود. زمانی که قرار است قفل برای یک دوره‌ی طولانی نگاه داشته شود، استفاده از سمافورها یا قفل‌های چرخشی مناسب است.